WRITING AUTOMATED TESTS FOR PYTHON APPLICATIONS

Internship Task: Task 3

Project Title: Student Result Management System

Programming Language: Python

Testing Framework: Pytest

Development Environment: Visual Studio Code

---

1. INTRODUCTION

Automated testing is an important part of software development. It helps developers verify whether an application works correctly without manually checking every function.

The objective of this project is to develop a Python application and implement automated test cases using the Pytest framework.

A Student Result Management System was developed to validate student marks, calculate average marks, assign grades and process student results.

Automated testing was performed to identify logical errors, verify functionality and improve application reliability.

2. OBJECTIVES

The main objectives are:

1. **To understand automated software testing.**
2. **To develop a Python application.**
3. **To implement unit testing using Pytest.**
4. **To perform positive and negative testing.**
5. **To identify and correct programming errors.**
6. **To perform boundary-value testing.**

7. **To improve software reliability and maintainability.**

3. TOOLS AND TECHNOLOGIES

Python: Used to develop the Student Result Management System.

Visual Studio Code: Used as the development environment for writing and executing Python programs.

Pytest: Used to create and execute automated test cases.

GitHub: Used for project version control and submission.

4. APPLICATION DEVELOPMENT

The application contains four major functions.

- **validate_marks(): Validates student marks.**
- **calculate_average(): Calculates the average of student marks.**
- **calculate_grade(): Assigns grades based on average marks.**
- **process_result(): Combines the functions and generates student results.**

Implementation Example

```
def calculate_average(marks):
    if not marks:
        raise ValueError("Marks list cannot be empty")

    for mark in marks:
        validate_marks(mark)
    return sum(marks) / len(marks)
```

Explanation:

The function calculates the average of student marks. Before performing the calculation, it checks whether the list is empty and validates every mark.

This prevents invalid data from producing incorrect results.

Screenshot 1: Insert the screenshot showing the application source code.

5. AUTOMATED TESTING USING PYTEST

Pytest was selected because it provides a simple and effective approach to automated testing.

The framework allows individual functions to be tested independently and identifies whether actual results match expected results.

The following command was used to execute the test cases:

```
python -m pytest -v
```

Example Test Case

```
def test_calculate_average():
    assert calculate_average([80, 90, 100]) == 90
```

Explanation:

The test supplies three marks: 80, 90 and 100.

The expected average is 90. The assertion checks whether the actual result matches the expected result.

If both values match, the test passes.

Screenshot 2: Insert the screenshot showing the automated test execution.

6. TYPES OF TESTING PERFORMED

6.1 Positive Testing

Positive testing verifies whether the application produces correct results for valid inputs.

Example:

Input: 85

Expected result: Valid marks.

6.2 Negative Testing

Negative testing verifies how the application handles invalid inputs.

Example:

Input: -10

Expected result: ValueError.

6.3 Boundary-Value Testing

Boundary-value testing checks values at the limits of the accepted range.

Examples:

- **0 marks**
- **50 marks**
- **75 marks**
- **90 marks**
- **100 marks**

This ensures that the application handles boundary conditions correctly.

6.4 Unit Testing

Unit testing verifies individual functions independently.

The validation, average calculation, grade calculation and result processing functions were tested separately.

7. BUG IDENTIFICATION AND CORRECTION

During the testing demonstration, a logical error was identified in the original grade calculation.

7.1 Before Correction

```
def calculate_grade(average):
    if average > 90:
        return "A"
    elif average > 75:
        return "B"
    elif average > 50:
        return "C"
    else:
        return "F"
```

Problem:

The greater-than operator (>) excluded exact boundary values.

For example, a student scoring exactly 90 received grade B instead of grade A.

7.2 After Correction

```
def calculate_grade(average):
    validate_marks(average)


    if average >= 90:
        return "A"
    elif average >= 75:
        return "B"
```

```
    elif average >= 50:
        return "C"
    else:
        return "F"
```

Improvement:

The greater-than-or-equal-to operator (>=) includes the boundary values in their correct grade categories.

7.3 Analytical Reasoning

The original implementation failed to handle exact grade boundaries correctly.

The corrected implementation ensures that students scoring exactly 90, 75 and 50 receive the appropriate grades.

This correction improves the logical accuracy of the application.

Automated boundary tests were used to verify the corrected implementation.

Screenshot 3: Insert the before-correction code.

Screenshot 4: Insert the corrected code.

Screenshot 5: Insert the successful boundary testing results.

8. TEST CASES AND RESULTS

A total of 15 automated test cases were implemented.

Test No.	Test Description	Expected Result
1	Valid marks	True
2	Invalid marks	ValueError
3	Average calculation	90

4	**Empty marks list**	**ValueError**
5	**Grade calculation**	**Correct grade**
6	**Invalid grade input**	**ValueError**
7	**Student result processing**	**Correct result**
8	**Empty student name**	**ValueError**
9	**Minimum marks**	**True**
10	**Maximum marks**	**True**
11	**Negative marks**	**ValueError**
12	**Non-numeric marks**	**TypeError**
13	**Blank student name**	**ValueError**
14	**Grade boundary testing**	**Correct grades**
15	**Grade boundary correction**	**Correct grades**

Testing Outcome

The test suite was executed using Pytest.

The expected final result was:

15 PASSED

This indicates that all implemented test cases passed successfully.

Screenshot 6: Insert the terminal screenshot displaying all 15 passed test cases.

9. CHALLENGES FACED

During the project, the following challenges were addressed:

1. **Understanding the purpose of automated testing.**

2. **Learning how to install and use Pytest.**
3. **Designing test cases for different inputs.**
4. **Handling invalid inputs using exceptions.**
5. **Identifying logical errors in grade calculation.**
6. **Verifying boundary conditions.**

These challenges improved my understanding of Python programming and software testing.

10. ANALYSIS AND LEARNING OUTCOMES

This project demonstrated that automated testing is essential for developing reliable applications.

Positive testing verified normal application behaviour, while negative testing checked error handling.

Boundary-value testing helped identify a logical error that ordinary test inputs might not reveal.

The before-and-after comparison demonstrated how a small change in a comparison operator could significantly affect application results.

The use of Pytest also reduced the need for repetitive manual checking and made it easier to verify changes.

Through this project, I gained practical knowledge of test-case design, assertions, exception handling and debugging.

11. CONCLUSION

The Python Automated Testing project was successfully developed using Python and Pytest.

The Student Result Management System demonstrated mark validation, average calculation, grade assignment and result processing.

Fifteen automated test cases were implemented to verify the application's functionality, including positive, negative and boundary-value scenarios.

A logical error in grade calculation was identified and corrected through testing.

Overall, this project improved my practical understanding of automated software testing and the importance of writing reliable and maintainable Python applications.

12. REFERENCES

1. **Python Official Documentation.**
2. **Pytest Official Documentation.**
3. **Visual Studio Code Documentation.**